算分第1次作业

郭劲豪 2200013146@stu.pku.edu.cn

习题1

1.2

(1)

无论a数组内的元素以何种顺序排列，我们对于每一个整数对$< i, j >, i < j$都要进行一次比较操作。

所以可知，最坏情况下要做$\frac{n(n-1)}{2}$次比较。

(2)

我们考虑按从大到小的顺序输入a数组中的元素，那么$\forall i, \forall j > i$都满足$A_j < A_i$,都需要进行一次交换，而在第二个for循环中，我们仅仅是将最后一个元素（也就是最小的元素）换到了第一个，其他的顺序不变，所以按照数学归纳法可知，一共要交换$\frac{n(n-1)}{2}$次，要求满足输入为按次序递减的A数组。

1.6

求解的问题为多项式求值，符号化的表示为：

> 输入多项式$P_0 + P_1x + P_2x^2 + \cdots + P_nx^n$，给出多项式系数和$x$的值，返回多项式的值。

乘法运算次数：$2n$次。因为计算x^i有一次乘法，累加求和有一次乘法。

加法运算次数：n次。

1.7

(1)

考虑S数组按照从大到小的顺序输入，此时程序中的每一行比较都会被执行，所以这是最坏情况，在这种情况下一共需要执行$2*(n-2)+1 = 2n-3$次比较。

(2)

考虑对于$i = 3 \ldots n$中的数，如果S_i是前最小或前第二小，则需要比较2次，否则只需要比较一次。

$$
\begin{aligned}
ans &= 1 + \sum_{i=3}^{n}(\frac{2}{i} * 2 + \frac{i-2}{i}) \\
&= 1 + \sum_{i=3}^{n}(1 + \frac{2}{i}) \\
&\approx n - 1 + 2\ln n - 3 \\
&= n - 4 + 2\ln n
\end{aligned}
$$

1.15

(1)

$f(n)=\Theta(n^2), g(n)=\Theta(n)$，所以$g(n)=O(f(n))$.

(2)

$f(n)=\Theta(n), g(n)=\Theta(n^2)$，所以$f(n)=O(g(n))$.

(3)

$f(n)=\Theta(n\log n), g(n)=\Theta(n^{1.5})$,所以$f(n)=O(g(n))$

(4)

$f(n)=\Theta(\log^2 n), g(n)=\Theta(\log n)$,所以$g(n)=O(f(n))$

(5)

考虑$\log(n!)=\sum_{i=1}^{n}\log n<n\log n$,又因为$g(n)=\Theta(n^{1.05})$,所以$f(n)=O(g(n))$

1.18

按照阶排序为:

$$
\begin{gathered}
n!, 2^{2n}, n2^n, \\
\log n^{\log n}=\Theta(n^{\log\log n}), \\
n^3, \\
n\log n=\Theta(\log(n!)), \\
n=\Theta(\log 10^n), \\
2^{\log\sqrt{n}}, 2^{\sqrt{2\log n}}, \\
\log n=\Theta(\sum_{k=1}^{n}\frac{1}{k}), \\
\log\log n
\end{gathered}
$$

1.19

(2)

由主定理可知:

$$T(n)=\Theta(n^2)$$

(4)

$$
\begin{aligned}
T(n) &= T(n-1)+\log 3^n \\
&= T(n-1)+n\log 3 \\
&= \dots \\
&= \log 3\times\frac{n(n+1)}{2} \\
&= \Theta(n^2)
\end{aligned}
$$

(6)

由主定理可知:

取$c=0.6$,则:

$$\frac{n^2}{2}\log\frac{n}{2} \le c \times n^2 \log n$$

所以可知：

$T(n) = \Theta(n^2 \log n)$

(8)

$$\begin{aligned} T(n) &= \log n + \log(n-1) + \cdots + 1 \\ &= \log n! \\ &= \Theta(n \log n) \end{aligned}$$

1.21

我们不妨设$T_i(n)$表示算法i在规模为n下的最差时间复杂度，则：

$T_1(n) = 5 \times T_1(\frac{n}{2}) + O(n)$
$T_2(n) = 2 \times T_2(n-1) + O(1)$
$T_3(n) = 9 \times T_3(\frac{n}{3}) + O(n^3)$

由主定理可知：

$$\begin{aligned} T_1(n) &= O(n^{\log_2 5}) \\ T_2(n) &= O(2^n) \\ T_3(n) &= O(n^3) \end{aligned}$$

所以最优的是算法A。

求解证明题

1.

证明：

$T(n) = 2T(\frac{n}{2}) + \frac{n}{\log n}$

考虑用递归树来解决，那么设$2^k = n$，则：

$$\begin{aligned} T(n) &= \frac{n}{k} + \frac{n}{2(k-1)} \times 2 + \ldots + \frac{n}{k-(k-1)} \\ &= n \times (\frac{1}{k} + \frac{1}{k-1} + \cdots + 1) \\ &= \Theta(n \ln \log n) \end{aligned}$$

代入可知：

$$\begin{aligned} T(1) &= 1 \\ 2T(\frac{n}{2}) + \frac{n}{k} &= 2 \times \frac{n}{2}(\frac{1}{k-1} + \frac{1}{k-2} + \cdots + 1) + \frac{n}{k} \\ &= n \times (\frac{1}{k} + \frac{1}{k-1} + \cdots + 1) = T(n) \end{aligned}$$

证毕。

2.

证明：考虑用递归树来解决，我们假设一共有k层，则：$2^k = n$.

将所有的值累加可以得到：$T(n) = 1 + 1 + 1 + \cdots + 1 = k = \log n$.

证毕。